Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

React Router

‹ Previous Next ›

What is React Router?

SHARE

React Router is a library that provides routing capabilities for React applications.

Routing means handling navigation between different views.

React Router is the standard routing library for React applications. It enables you to:

- Create multiple pages in your single-page application
- Handle URL parameters and query strings
- Manage browser history and navigation
- Create nested routes and layouts
- Implement protected routes for authentication

Without a router, your React application would be limited to a single page with no way to navigate between different views.

W3Schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

install the package:

```
npm install react-router-dom
```

Wrap Your App with BrowserRouter

Your application must be wrapped with the `BrowserRouter` component to enable routing:

```
function App() {
  return (
    <BrowserRouter>
      {/* Your app content */}
    </BrowserRouter>
  );
}
```

Create Views

To demonstrate routing, we'll create three pages (or views) in our application: Home, About, and Contact:

We will create all three views in the same file for simplicity, but you can of course split them into separate files.

Example

Get your own React.js Server

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
function About() {
  return <h1>About Page</h1>;
}

function Contact() {
  return <h1>Contact Page</h1>;
}
```

REMOVE ADS

Basic Routing

React Router uses three main components for basic routing:

- `Link` : Creates navigation links that update the URL
- `Routes` : A container for all your route definitions
- `Route` : Defines a mapping between a URL path and a component

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Example

Note that we need to import `BrowserRouter, Routes, Route, Link` from 'react-router-dom'.

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function Contact() {
  return <h1>Contact Page</h1>;
}

function App() {
  return (
    <BrowserRouter>
      {/* Navigation */}
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link> |{" "}
        <Link to="/contact">Contact</Link>
      </nav>

      {/* Routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Run Example »

In this example:

- `BrowserRouter` wraps your app and enables routing functionality
- `Link` components create navigation links
- `Routes` and `Route` define your routing configuration

Nested Routes

You can have a `Route` inside another `Route`, this is called nested routes.

Nested routes allow you change parts of the page when you navigate to a new URL, while other parts is not changed or reloaded, almost like having a page within a page.

Let's use the example above, and add two new components that will be rendered inside the `Products` component.

One called `CarProducts` and one called `BikeProducts`:

Example

Note that we also need to import the `Outlet` component from 'react-router-dom'.

```
import { BrowserRouter, Routes, Route, Link, Outlet } from 'react-router-dc

function Home() {
  return <h1>Home Page</h1>;
}

function Products() {
  return (
    <div>
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
      </nav>
      <Outlet />
    </div>
  );
}

function CarProducts() {
  return (
    <div>
      <h2>Cars</h2>
      <ul>
        <li>Audi</li>
        <li>BMW</li>
        <li>Volvo</li>
      </ul>
    </div>
  );
}

function BikeProducts() {
  return (
    <div>
      <h2>Bikes</h2>
      <ul>
        <li>Yamaha</li>
        <li>Suzuki</li>
        <li>Honda</li>
      </ul>
    </div>
  );
}

function Contact() {
  return <h1>Contact Page</h1>;
}
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
        <nav>
          <Link to="/">Home</Link> |{" "}
          <Link to="/products">Products</Link> |{" "}
          <Link to="/contact">Contact</Link>
        </nav>

        {/* Routes */}
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/products" element={<Products />}>
            <Route path="car" element={<CarProducts />} />
            <Route path="bike" element={<BikeProducts />} />
          </Route>
          <Route path="/contact" element={<Contact />} />
        </Routes>
      </BrowserRouter>
    );
  }
```

Run Example »

Important notes about the example above:

1. **Outlet:**
   The `<Outlet />` element in the `Products` component specifies where to render the child route's content.

2. **Routes:**
   The `Routes` element contains the routes to `CarProducts` and `BikeProducts` as child routes of the `Products` parent route.

3. **URL Structure:**
   The URL structure is relative to the parent route's path. For example:
   - When you navigate to `'/products/car'`, the `CarProducts` component is rendered.

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Style Active Links

There is a special version of the `Link` component called `NavLink` that knows whether the link's URL is "active" or not.

The `NavLink` is especially useful for:

- Navigation menus
- Breadcrumbs
- Tabs

A `NavLink` is considered active if the current URL matches its `to` prop.

The `NavLink` component makes it easier to style active links.

Take the basic example from above, and add styles for active links using `NavLink`:

Example

Create a new element called `navLinkStyles` and replace `<Link>` with `<NavLink>` in `App`.

Note that we also need to import the `NavLink` component from 'react-router-dom'.

```
import { BrowserRouter, Routes, Route, NavLink } from 'react-router-dom';

// Style function for active links
const navLinkStyles = ({ isActive }) => ({
  color: isActive ? '#007bff' : '#333',
  textDecoration: isActive ? 'none' : 'underline',
  fontWeight: isActive ? 'bold' : 'normal',
  padding: '5px 10px'
});

function Home() {
  return <h1>Home Page</h1>;
}
```

W3Schools app NEW iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
function Contact() {
  return <h1>Contact Page</h1>;
}

function App() {
  return (
    <BrowserRouter>
      {/* Navigation with NavLink for active styling */}
      <nav style={{ marginBottom: '20px' }}>
        <NavLink to="/" style={navLinkStyles}>Home</NavLink> |{" "}
        <NavLink to="/about" style={navLinkStyles}>About</NavLink> |{" "}
        <NavLink to="/contact" style={navLinkStyles}>Contact</NavLink>
      </nav>

      {/* Routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Run Example »

URL Parameters

URL parameters are variables that you can add to your route paths. They are often used to pass data between components.

In the path `http://localhost:5173/customer/Tobias`, the URL parameter is `Tobias`.

Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

Here's a simple example with a greeting page that can say hello to different customers:

Example

```
import { BrowserRouter, Routes, Route, Link, useParams } from 'react-router

function Info() {
  const { firstname } = useParams();
  return <h1>Hello, {firstname}!</h1>;
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/customer/Emil">Emil</Link> |
        <Link to="/customer/Tobias">Tobias</Link> |
        <Link to="/customer/Linus">Linus</Link>
      </nav>

      <Routes>
        <Route path="/customer/:firstname" element={<Info />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Run Example »

In this example:

- `:firstname` in the route path is the URL parameter

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

- You can use any name in the URL, and the greeting will work!, try this
  http://localhost:5173/customer/John

Previous Sign in to track progress Next

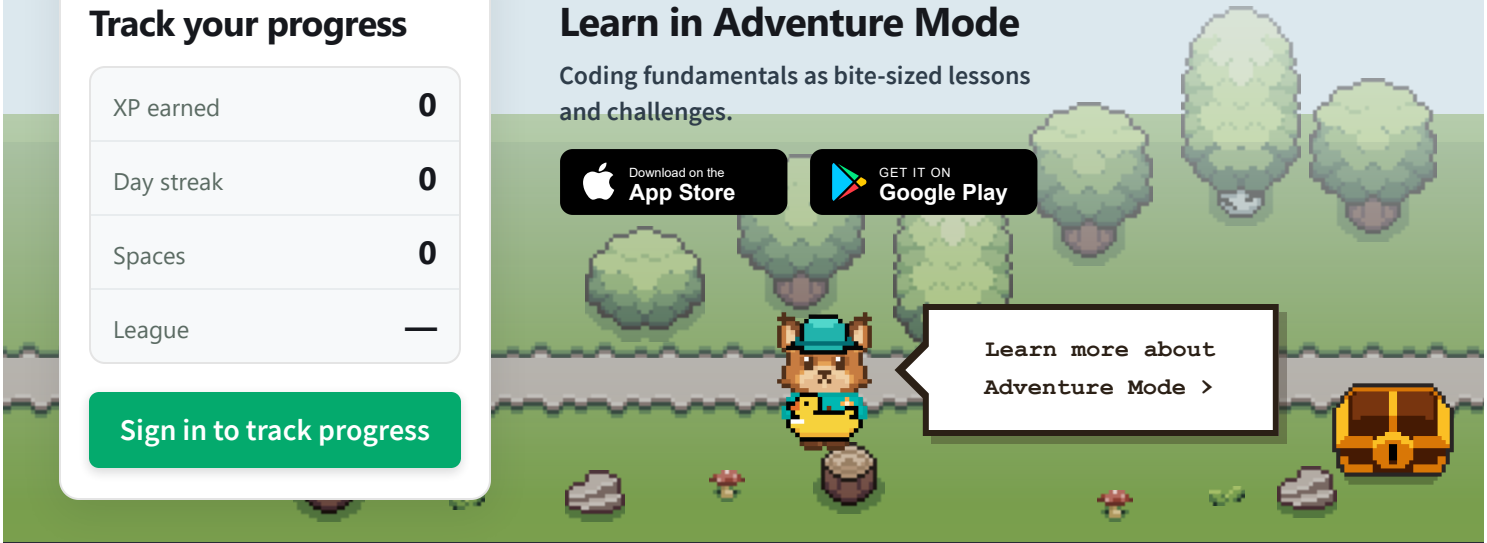
Build skills.
Get hired.
$95 per paid course
one-time payment
Exam and certificate included.
Get started

REMOVE ADS

Tutorials Exercises
W3Schools app iOS & Android NEW
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

FREE ACCOUNT
Track your progress
XP earned 0
Day streak 0
Spaces 0
League —
Sign in to track progress

NEW
Learn in Adventure Mode
Coding fundamentals as bite-sized lessons and challenges.
Download on the App Store
GET IT ON Google Play



PLUS SPACES

Tutorials Exercises
W3Schools app NEW
iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS
PRACTICE CONTACT US

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.